

Day 71

26/05/2025

why use mvvm ^{unit} (testing).

- * Clear separation of concerns
- * Easier to maintain and test.
- * Support WPF data binding
- * No code behind logic needed for UI actions.

~~Key features~~ key features in MVVM

1. Notify property changed

* Enables viewmodel properties to notify the view when their values change.

* Essential for updating UI automatically on data changes.

* Implement by raising property changed event in property setters.

2) ICommand

- * Handles user action like button clicks in the viewmodel.
- * Keep event logic out of the view code behind.
- * Supports enabling / disabling controls based on conditions (can execute)

3) ObservableCollection

- * A collection that automatically notifies the view of any changes (add / remove)
- * Ideal for binding lists, grids or any dynamic collections.

4) data Binding

- * Connects view properties to view model properties.

- * support one way and one time binding modes.

- * Enables synchronization between UI and data automatically

5) Value Converter

- * Transforms data between viewmodel and view during binding.

- * useful for formatting converting type or conditional UI logic.

Helper window model C# + .

```
public class RelayCommand : ICommand
```

```
{  
    private Action _action;
```

```
    public RelayCommand(Action action)
```

```
{  
        _action = action;
```

```
}
```

```
    canExecute()
```

```
    {  
        return true;
```

```
    }  
    Execute()
```

```
    {  
        _action.Invoke();  
    }  
}
```


View model - RelayCommand

```
public RelayCommand RelayCommand { get; set; }
```

```
RelayButton = new RelayCommand(Relay);
```

```
public void RelayCommand()
```

```
{
```

```
}
```

```
command =
```

```
<Button x:Name="Button" {Binding="RelayButton"}
```

model: Notify

```
public class Notify : INotifyPropertyChanged
```

```
{
```

```
public event PropertyChangedEventHandler PropertyChanged
```

```
protected void OnPropertyChanged(string propertyName)
```

```
{
```

```
PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName))
```

public class module : Notify

{
private String name;
public String Name

{

get

{

return name

}

set

{

name = value

onproperty changed ("Name");

not need

caller member name
is called member

}